Software brief: dosing station interface

What the software side has to do, in one read. Hardware: ESP32 + load cell, one auger, PC over USB.

1. The idea

An operator uses a **PC interface** to enter a recipe and a sample ID, check that the machine is ready, and start a dose. The **ESP32** does the real-time work: it weighs, drives the auger, verifies the result and reports back. The interface shows everything live and saves a log of every dose.

- **Golden rule:** dosing is driven by the weight reading, never by time.
- **START and STOP come from the PC only.** The machine has just a power switch and a TARE button.

2. Who does what

ESP32 firmware (C++/Arduino)	PC interface (Python suggested)
Read the load cell (HX711), average and filter it Drive the stepper (STEP, DIR, EN) and the vibration module Read the cup sensor, hopper level sensor, TARE button, 12 V sense Run the dosing loop and the safety checks Show key info on the LCD and a color on the RGB LED Send status and results over USB serial	Recipe and sample ID form Pre-start checklist, START enabled only when all green Live weight, phase and chart during the dose Result card: target, measured, deviation, status Save the log (CSV or SQLite) and show the history Repeatability stats (mean error, spread, rejected count)

3. ESP32 pins

Part	Pins	Notes
HX711 load cell	DT = 16, SCK = 17	Calibrate with a known mass, tare before every dose
Stepper driver	STEP = 25, DIR = 26, EN = 27	Microstepping set on the driver
Vibration module	IN = 14	On only to break bridging, off in the fine phase
IR cup sensor	GPIO 34	LOW = cup present
I2C bus	SDA = 21, SCL = 22	LCD 20x4 (usually 0x27) and ToF sensor VL53L0X (0x29)
TARE button	GPIO 4	To GND, internal pull-up. Ignored during a dose
RGB LED	R = 18, G = 19, B = 23	Check common cathode or anode
12 V sense	GPIO 33	HIGH = machine power switch ON

4. USB serial protocol

115200 baud, **one JSON message per line** (ends with a newline). Same format both ways, so it is easy to test with any serial terminal.

```
PC -> ESP32
{"cmd":"start","recipe_id":"R-001","material_id":"MAT-A","sample_id":"B01-S03","target_g":10.0,"tol_g":0.1}
{"cmd":"stop"}     {"cmd":"tare"}     {"cmd":"status"}

ESP32 -> PC
{"evt":"status","state":"IDLE","weight_g":0.0,"cup":true,"hopper_ok":true,"power_ok":true,"tare_ok":true}
{"evt":"dose","phase":"FAST","weight_g":4.2,"t_ms":1500}              (every 100 to 200 ms while dosing)
{"evt":"result","sample_id":"B01-S03","target_g":10.0,"measured_g":10.1,"dev_g":0.1,"status":"ACCEPTED",
 "overshoot_g":0.3,"trim_steps":2,"t_start":"...","t_end":"..."}
{"evt":"error","code":"NO_CUP"}     (codes: NO_CUP, NO_FLOW, HOPPER_EMPTY, POWER_OFF, TIMEOUT, ABORTED)
```

5. Machine states and LED colors

IDLE (blue) → **CHECKS** → **DOSING** (orange: FAST, SLOW, FINE, TRIM) → **SETTLE** (wait about 0.5 s) → **VERIFY** → **DONE** (green = ACCEPTED, red = REJECTED) or **ERROR** (red). The ESP32 stops the motor by itself on any error, and also if the PC link is lost during a dose.

6. Dosing logic (firmware)

- **Speed by remaining weight:** fast until about 75% of the target, slow until about 95%, then fine with a few microsteps at a time.
- **Stop threshold in grams:** stop at *target minus expected overshoot* (powder still falling). Overshoot = moving average of the last 3 to 5 runs.
- **Trim loop:** if still below target minus tolerance, add 1 to 3 microsteps, wait, measure, repeat (maximum attempts, then flag it).
- **Never remove powder:** always approach the target from below. Over the tolerance means REJECTED.
- **No flow:** auger turning but weight not rising means NO_FLOW (empty hopper or bridging). Turn the vibration on, then flag the error if it persists.
- **Checks before a dose:** power ON, cup present, weight stable after tare, hopper level OK. The PC also checks that the recipe is valid and the **sample ID is unique**.

7. Interface screens (keep it simple)

- **Setup:** recipe ID, material ID, target (g), tolerance (g), sample ID, plus the pre-start checklist with green ticks and the START button.
- **Live:** big weight number, current phase, progress bar toward the target, weight-versus-time chart with the three phases in different colors, STOP button.
- **Result:** target, measured, deviation, status (green or red), then "next sample".
- **History:** table of all runs (also the rejected and failed ones), export to CSV, stats: mean error, standard deviation, rejected count, overshoot over runs.
- The recipe format must accept **several lines** (one per material) like `lines:[{material_id, target_g, tol_g}]`, even though the demo uses one.

8. Log columns (one row per dose)

`start_time, end_time, recipe_id, material_id, sample_id, target_g, measured_g, deviation_g, tol_g, status, overshoot_g, trim_steps, error, method`. Rejected and failed runs stay in the log, never overwritten.

9. Suggested order of work

- **First, a mock ESP32:** a small Python script that answers the same JSON messages with fake weights. Build and test the whole interface with it, because the real machine only exists at the makeathon (16 Oct).
- Then: serial link and result card, CSV log, pre-start checks, live chart, history and stats.
- Firmware in parallel: read and calibrate the load cell, drive the stepper, then the dosing loop and safety checks.
- Bonus if time allows: learning the overshoot from past runs, comparison table against manual dosing, a second material line.